

lab3

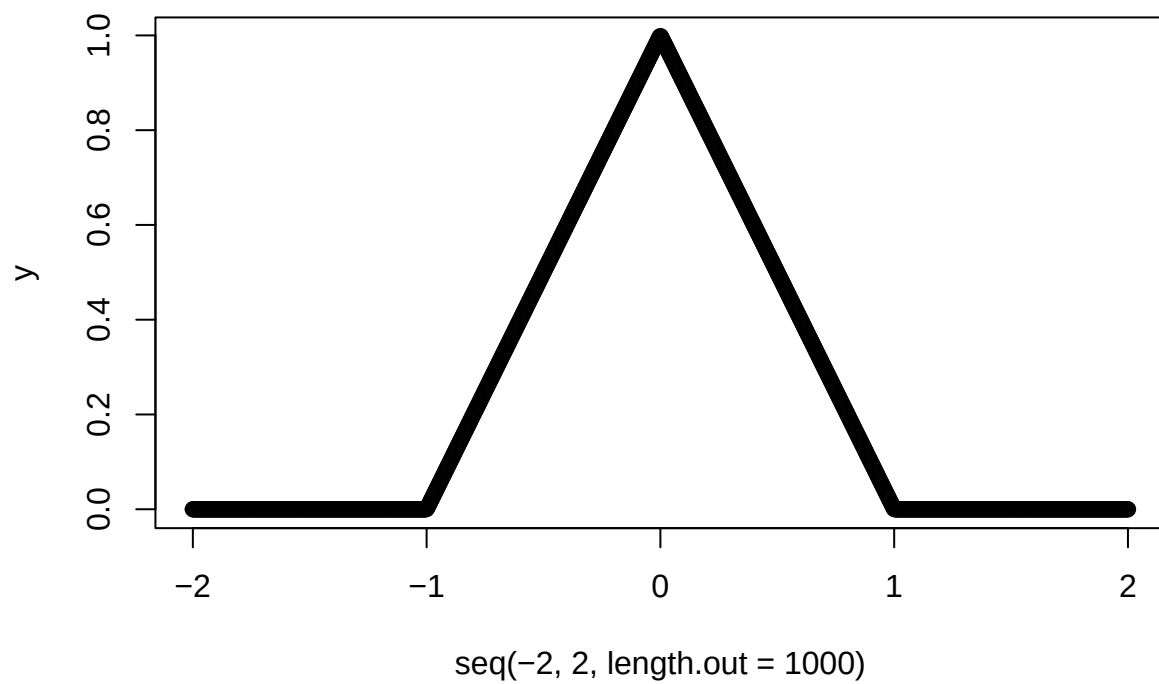
2026-02-05

Q1

a

Lets choose a uniform distribution instead of a normal envelope for simplicity. The triangle density needs an envelope in $[-1, 1]$, elsewhere it is 0. It has a maxima of 1 so we need to scale our uniform envelope by 2. Our waste will be $1/2$, we enclose a function with area 1 with a function of area 2.

```
tri_fun <- function(x) {  
  
  if (x < -1) {  
    return(0)  
  } else if (x <= 0) {  
    return(x+1)  
  } else if (x <= 1) {  
    return(1-x)  
  } else {  
    return(0)  
  }  
}  
  
y <- sapply(seq(-2, 2, length.out = 1000), tri_fun)  
plot(seq(-2, 2, length.out = 1000), y)
```



```
set.seed(123)
n <- 10000
rejection_count <- 0
sample <- c()

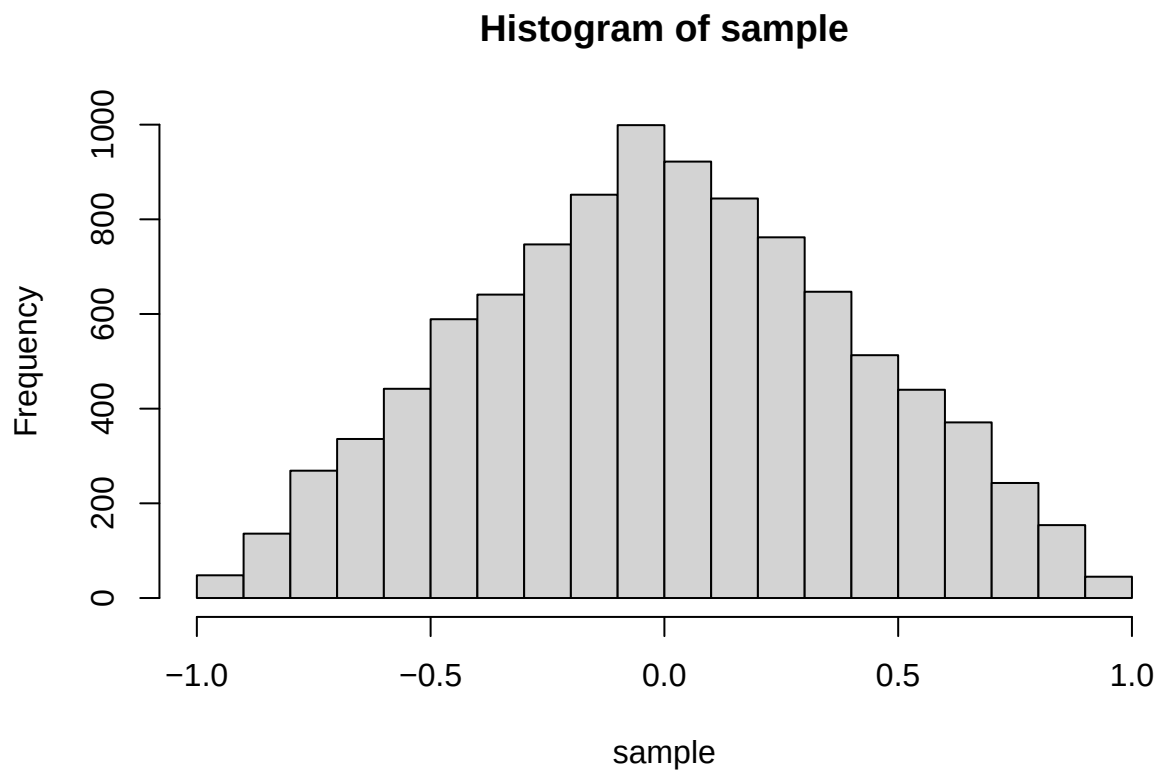
while(length(sample) < n){
  U1 <- runif(1, -1, 1)
  U2 <- runif(1, 0, 1)

  if (tri_fun(U1) > U2) {
    sample <- c(sample, U1)
  } else {
    rejection_count <- rejection_count + 1
  }
}

waste <- rejection_count / (n + rejection_count)
print(waste)
```

```
## [1] 0.4956118
```

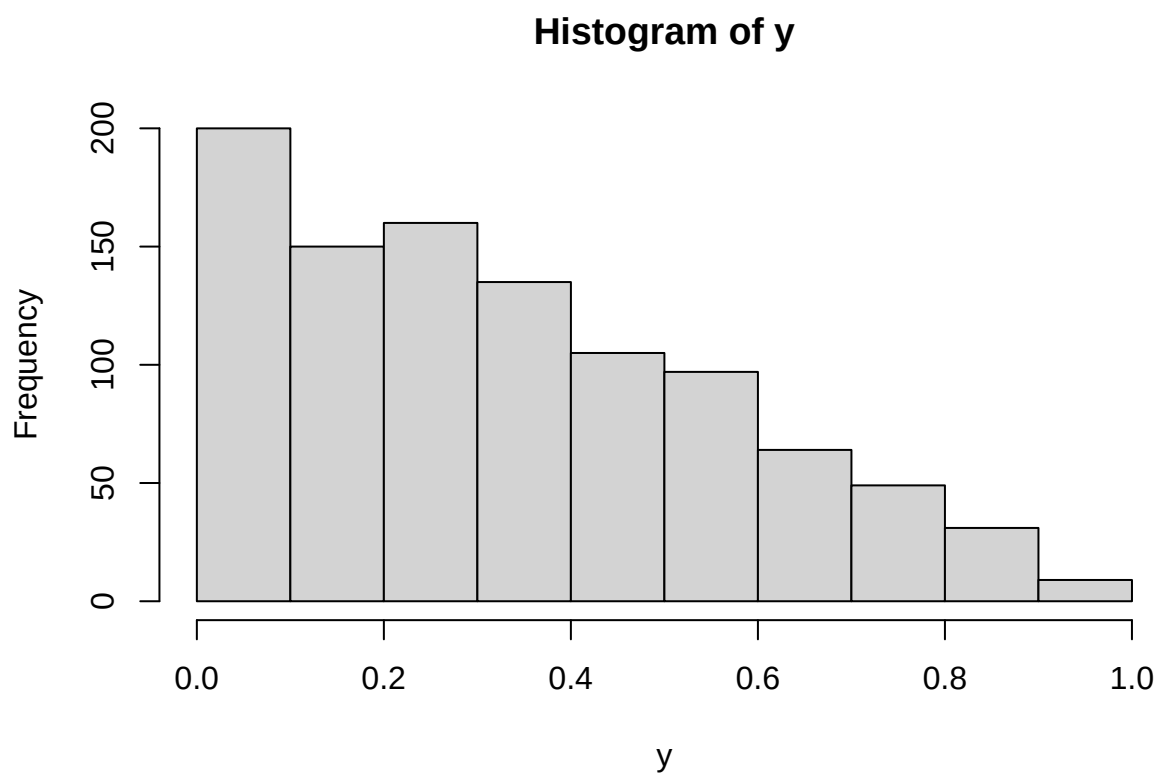
```
hist(sample)
```



Waste matches the theoretical waste.

b

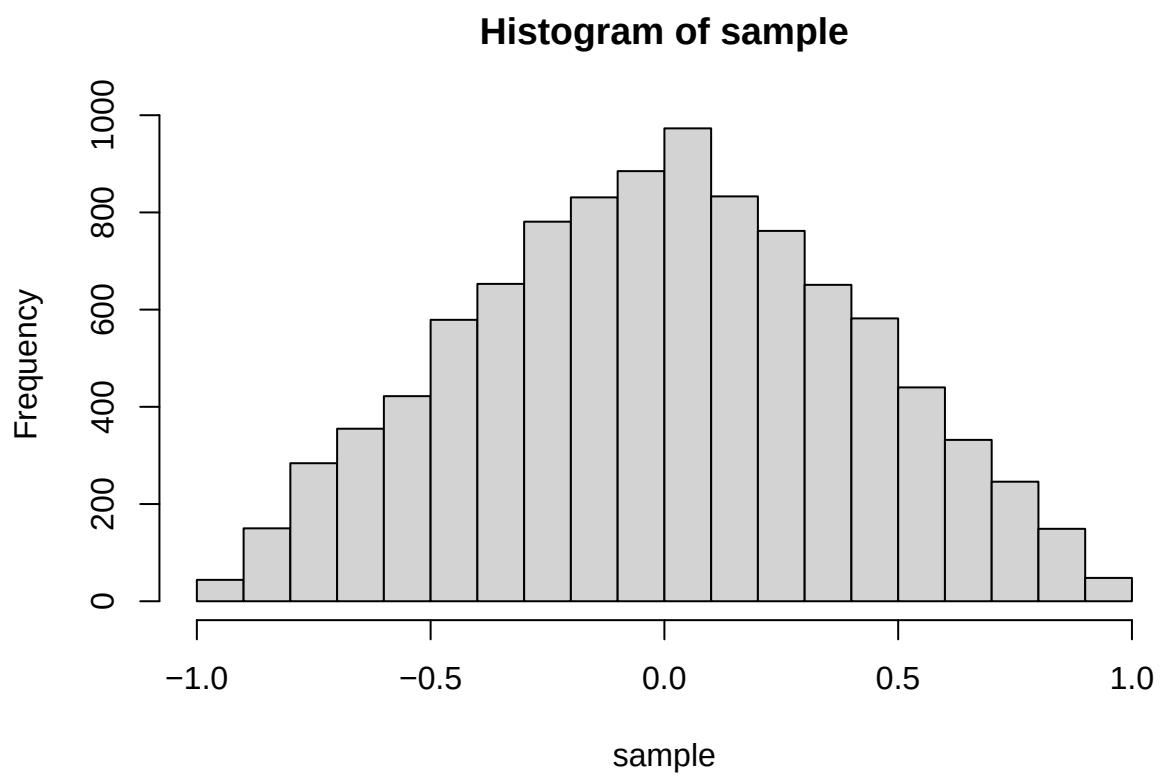
```
#code from slides  
u <- runif(1000)  
y <- 1 - sqrt(1-u)  
hist(y)
```



```
set.seed(1234)
sample <- c()
for (i in 1:n) {
  # choosing which distribution to sample from with a 50/50 chance
  distribution_multiplier <- sign(runif(1, -1, 1))
  u <- runif(1)
  y <- 1 - sqrt(1-u)

  sample <- c(sample, y*distribution_multiplier)
}

hist(sample)
```

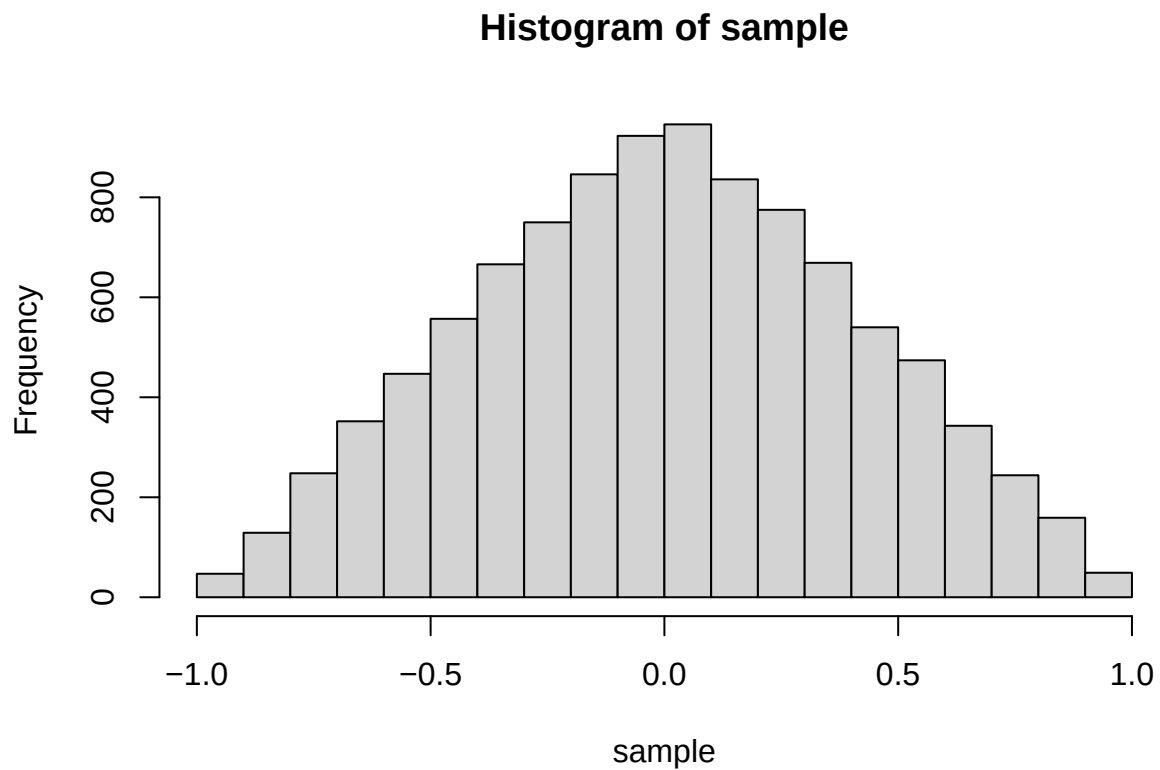


c

```
set.seed(12345)
sample <- c()
for (i in 1:n) {
  u1 <- runif(1)
  u2 <- runif(1)

  sample <- c(sample, u1 - u2)
}

hist(sample)
```



d

U1-U2 uses $2 \times n$ random numbers. The inverse cdf also uses $2 \times n$ but performs more calculations. Rejection sampling has some variance, it uses $2 \times n$ on average with a lazy chosen envelope, with a better envelope it is the clear winner.

Q2

a

```
system.time({  
  
n <- 100000  
  
sample1 <- c()  
sample2 <- c()  
  
for (i in 1:n) {  
  A <- runif(1, 0, 2*pi)  
  U <- runif(1)  
  Y <- sqrt(-2 * log(U))  
}
```

```

X1 <- Y * cos(A)
X2 <- Y * sin(A)

sample1 <- c(sample1, X1)
sample2 <- c(sample2, X2)

}

})

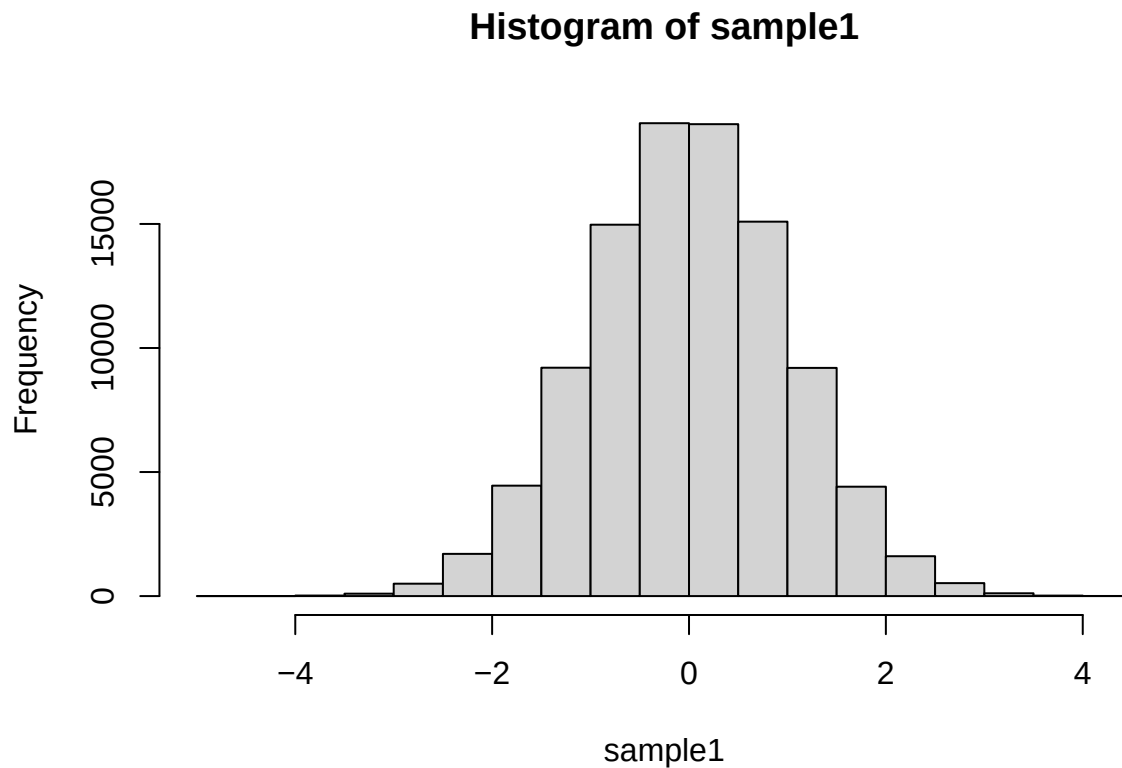
```

```

##      user  system elapsed
##  24.30    6.50    30.89

```

```
hist(sample1)
```



I use a smaller n as my computer is too slow. Box muller gives us a standard normal $X \sim N(0, I)$ To get $N(\mu, \Sigma)$ we apply $Y = A^T X + \mu$ where A^T comes from the cholesky decomposition of Σ

```

A_t <- chol(matrix(c(0.6, 0, 0, 0.6), 2, 2))
A_t %*% t(A_t)

```

```

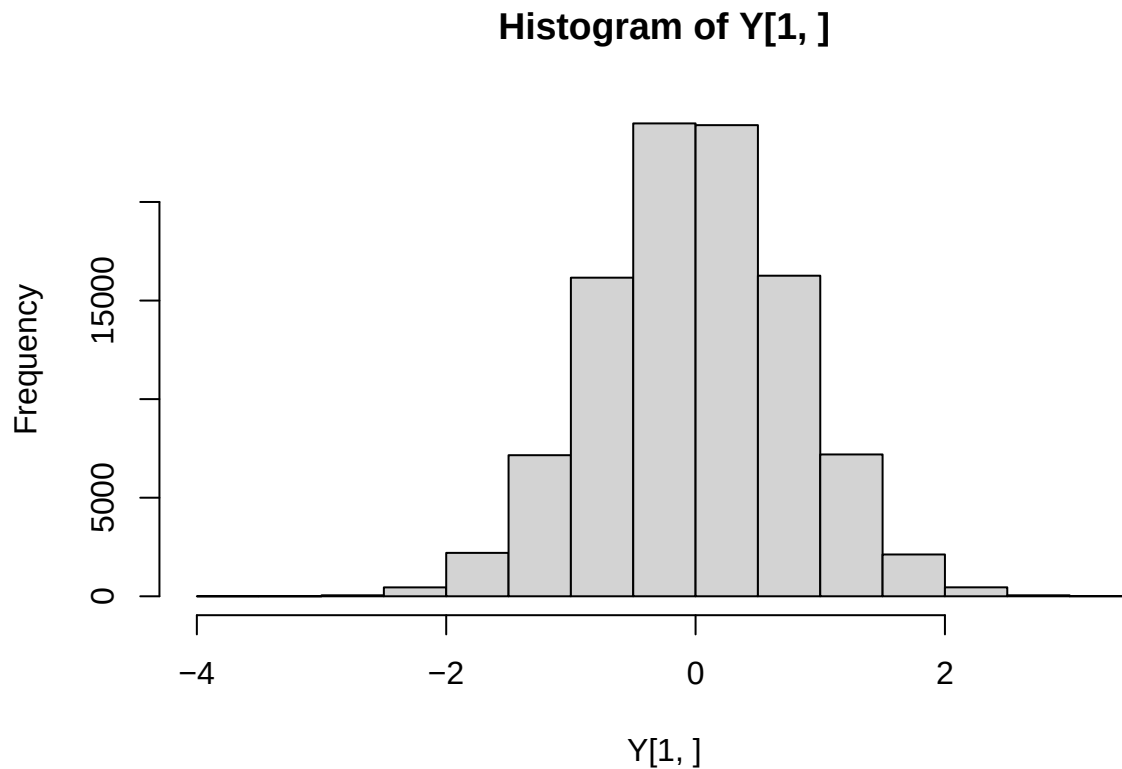
##      [,1] [,2]
## [1,]  0.6  0.0
## [2,]  0.0  0.6

```

```
X <- rbind(sample1, sample2) |> as.matrix()
dim(X)
```

```
## [1]      2 100000
```

```
# 0 mean
Y <- A_t %*% X
hist(Y[1, ])
```



```
## b The variance is 0.6 so we take the square root.
```

```
system.time({
  x1 <- rnorm(n, 0, sqrt(0.6))
  x2 <- rnorm(n, 0, sqrt(0.6))
  X <- rbind(x1, x2)
})
```

```
##      user  system elapsed
##    0.00    0.00    0.03
```

This was much faster than in a.

c

```
library(mvtnorm)

normal_bivariate <- function(n) {

  X <- matrix(0, 2, n)

  for (i in 1:n) {

    U <- runif(1)

    if (U < 0.5) {
      X[, i] <- rmvnorm(1, c(0, 0), diag(2) * 0.6)
    } else {
      X[, i] <- rmvnorm(1, c(1.5, 1.2), diag(2) * 0.5)
    }

  }

  library(ggplot2)

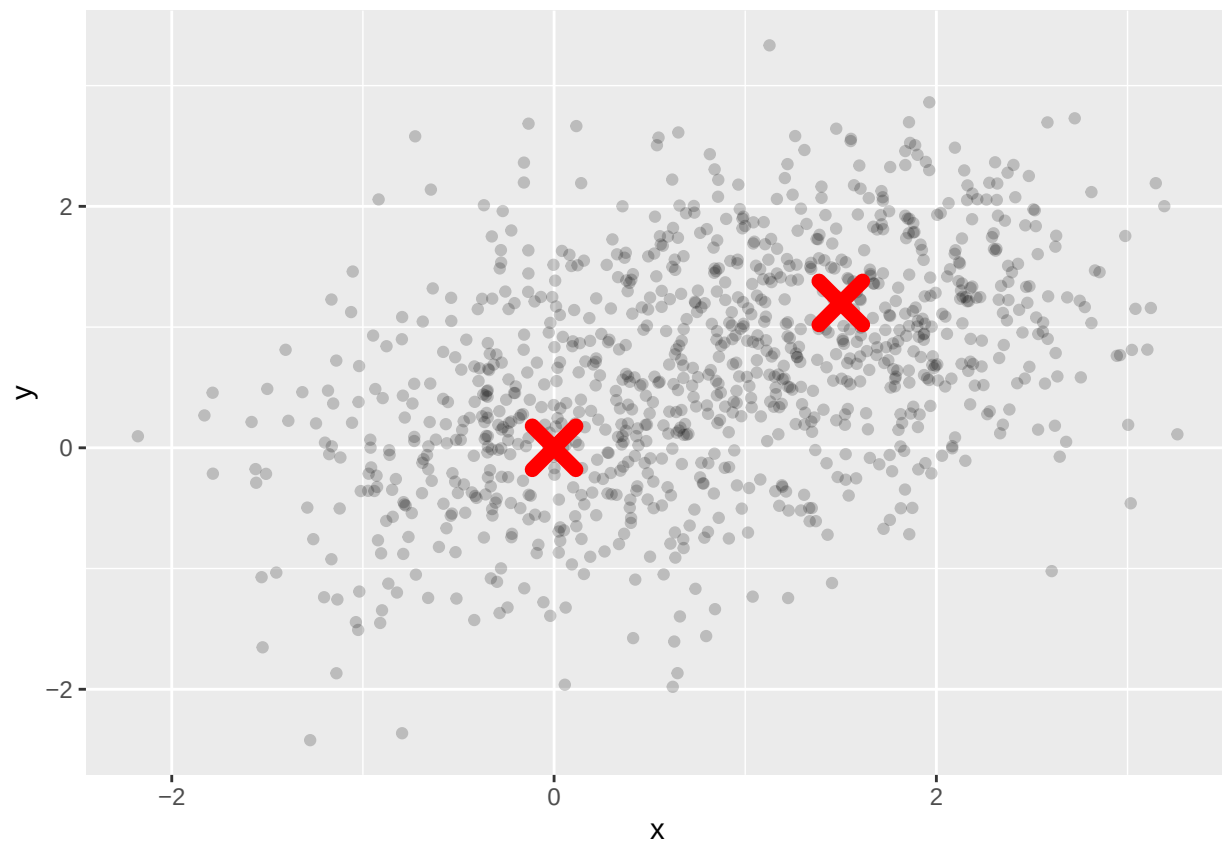
  df_main <- data.frame(x = X[1, ], y = X[2, ])
  df_points <- data.frame(x = c(0, 1.5), y = c(0, 1.2))

  ggplot(df_main, aes(x = x, y = y)) +
    geom_point(alpha = 0.2) +
    geom_point(data = df_points, aes(x = x, y = y),
              shape = 4,
              color = "red",
              size = 5, stroke = 4)

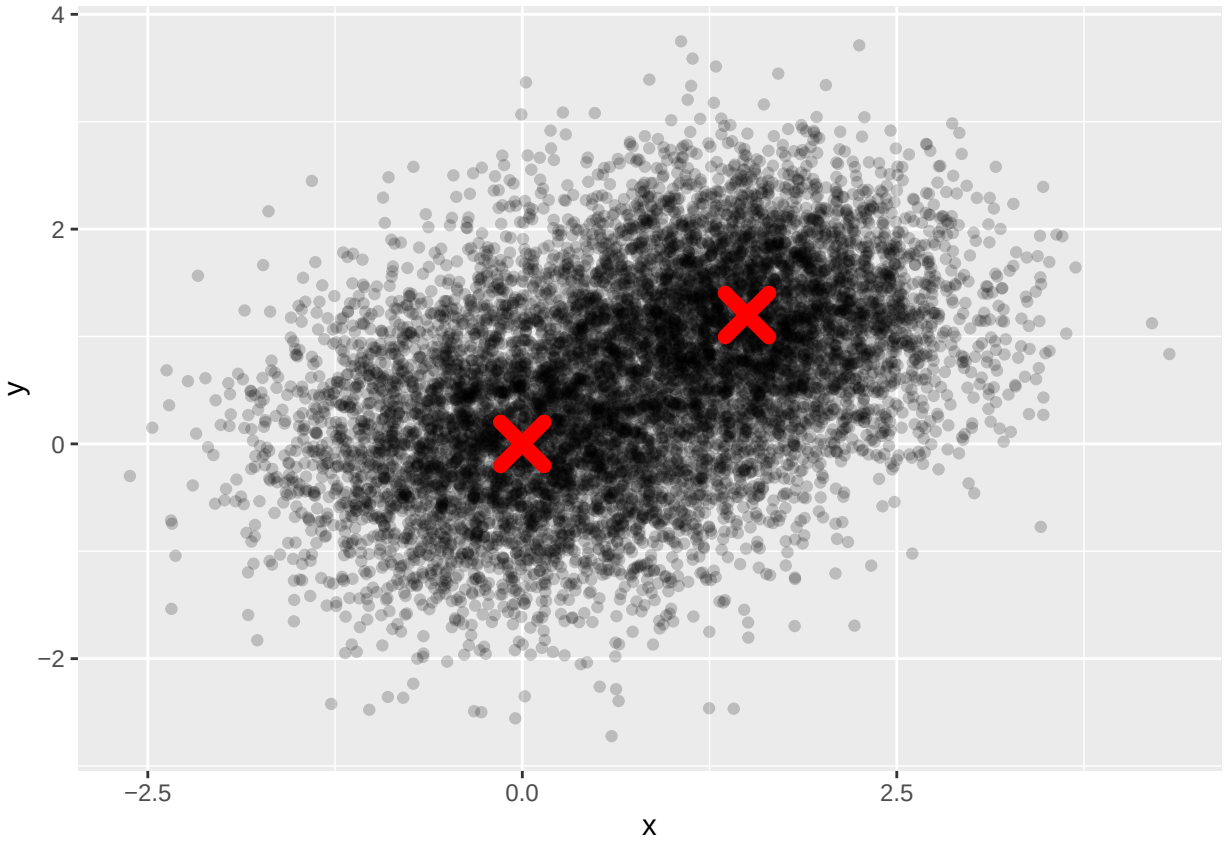
  }

  set.seed(12345)
  normal_bivariate(1000)
```

```
## Warning: package 'ggplot2' was built under R version 4.4.3
```



```
set.seed(12345)  
normal_bivariate(10000)
```



Here with a larger n we more clearly see that the density of points drops off further from the means, thanks to the α in the plot. We see 2 circles centered around the means which is to be expected.